Learning-Based Self-Healing Agents for Fault Detection and Recovery in Microservice Systems

Eamon Cullimore
DCU School of Computing
Dublin City University
Dublin, Ireland
eamon.cullimore2@mail.dcu.ie
A21360701

Hadrian Ging Hin Chong
DCU School of Computing
Dublin City University
Dublin, Ireland
hadrianginghin.chong2@mail.dcu.ie
A00053080

***Abstract*—Microservice architectures have become the dominant paradigm for developing scalable cloud-native applications. However, their distributed nature introduces significant challenges for fault detection, root-cause analysis, and service recovery. Traditional rule-based approaches rely heavily on predefined recovery policies and manual intervention, limiting their ability to respond effectively to complex runtime failures. This paper presents an AI-driven self-healing framework for microservice systems that integrates graph-based anomaly detection, reinforcement learning-based recovery, adaptive recovery execution, automated verification, and large language model (LLM) explainability into a unified operational pipeline. Distributed execution traces collected using Apache SkyWalking are transformed into graph representations and analysed using a Gated Graph Neural Network (GGNN) with Deep Support Vector Data Description (Deep SVDD). Trained exclusively on normal execution traces, the proposed anomaly detector achieved an AUC-ROC of 0.947 and a Precision of 0.968 on the DeepTraLog benchmark. A reinforcement learning recovery policy, trained using supervised pretraining followed by the REINFORCE policy-gradient algorithm, achieved 96.5% recovery-action selection accuracy across 14 representative fault categories. The complete framework was deployed on the Train-Ticket microservice benchmark, where it successfully detected and recovered representative runtime faults through adaptive recovery execution and automated verification. Experimental results demonstrate that integrating graph neural networks, reinforcement learning, and deployment-aware recovery provides an effective and practical solution for autonomous self-healing in cloud-native microservice environments.**

***Index Terms*—microservices, fault detection, self-healing, graph neural networks, reinforcement learning, distributed tracing, AI agents**

I. Introduction

Modern application development has undergone a substantial shift away from monolithic architectures, in which all functionality is bundled into a single codebase that becomes increasingly difficult to maintain, test, and redeploy as business requirements evolve [1]–[3]. In a monolithic system, even a small change to one module requires rebuilding and redeploying the entire application, and any single component failure can bring down the whole system [2], [4]. To address these limitations, the industry has increasingly adopted microservice architectures, which decompose applications into small, independently deployable services that communicate over lightweight protocols such as HTTP/REST or APIs [1], [3], [4]. This change improves scalability, fault isolation, and development agility by allowing teams to build, test, and scale individual services independently [1], [3]. However, this huge shift in design introduces substantial operational complexities. Because a single system operation may traverse dozens of independently deployed services, system concerns such as service communication, network latency, and data consistency become critical engineering problems rather than implementation details [3]–[5]. Tracing the root cause of a fault or performance degradation across this network of services is considerably more difficult than debugging a single-process application. Microservice instances are created and destroyed dynamically, run across large numbers of nodes, and communicate through complex, asynchronous call chains, all of which introduce uncertainty into the relationship between observed symptoms and underlying causes [4]. Prior work has also shown that microservice-based systems, despite their benefits, bring additional operational overhead in the form of security management, monitoring, and testing across many independent services [2], [5]. To mitigate these challenges, current industry and research practice relies heavily on cloud-native orchestration frameworks, automated delivery pipelines, and observation tooling such as:

- **Monitoring and Observations:** Organizations employ centralized logging, distributed tracing, and monitoring platforms to gain visibility into the health and performance of individual microservices to identify faults more quickly than would be possible through manual searching [3].
- **Continuous Health Oversight:** Monitoring pipelines further allow operators to continuously oversee the status of each deployed microservice, rather than relying solely on reactive incident response, an approach considered essential for maintaining reliability at large scale [3].
- **Service Meshes:** Technologies such as Istio are widely used to manage the complexity of microservice-to-microservice communication, offering built-in support for traffic routing, distributed tracing, fault tolerance, and network-level observability without requiring changes to individual services [2], [3].

- **Automated Resilience Testing:** Development teams increasingly embed automated functional and integration testing into continuous integration/continuous delivery (CI/CD) pipelines, improving the likelihood of catching defects before deployment and ensuring that a localized failure does not cascade into a full application outage [1].

Despite these mechanisms, evidence indicates that microservice adoption does not eliminate operational burden so much as relocate it. Existing monitoring and remediation workflows still depend substantially on threshold-based alerting and manual diagnostic effort, current debugging of microservice systems is reported to rely largely on operator experience and manual log inspections rather than automated root-cause analysis [4]. These approaches struggle to capture the complex, multi-service fault-propagation patterns evident in large microservice deployments, and the resulting dependence on manual intervention for diagnosis and remediation contributes to a slow mean time to recovery (MTTR) that only worsens as the number of services grow [4].

The remainder of this paper is structured as follows. Section II explains the background and motivations for this research. Section III reviews related work. Section IV describes the system methodology. Section V presents experiment setup and environment. Section VI analyses the results. Section VII concludes the paper with final observations. Section VIII contains the AI declaration as required.

II. BACKGROUND AND MOTIVATIONS

A. Monolithic Architecture

Industry applications have traditionally been developed following a monolithic architecture, in which all functional modules are packaged and deployed as a single logical application [6]. This design is straightforward to build, test, and deploy when an application is small, since all internal communication occurs through in-process function calls rather than network requests [4], [6]. However, as functionality and codebase size grow, monolithic systems become increasingly difficult to maintain as a change to one module risks unpredictable side effects elsewhere in the system. Start-up and deployment times lengthen, and the entire application must be rebuilt and redeployed even for isolated updates [2], [4]. Scaling is similarly complex, since the only option is to replicate the whole application rather than the specific component under load [4].

B. Microservice Architecture

Microservice architecture has emerged as a response to these limitations. Rather than a single deployable unit, an application is broken down into a set of small, independently deployable services, each responsible for a narrow business service and communicating over lightweight protocols such as HTTP/REST or the use of application programming interfaces APIs [4], [5]. Each service can be developed, tested, scaled, and deployed independently, using its own technology stack and private data store [2], [4]. This independence gives organisations more concise horizontal scalability, fault isolation (the failure of one service does not bring down the whole system) and more autonomous release cycles for individual teams [2], [5].

C. Operational Challenges of Microservice Decomposition

These benefits, however, are not without cost. Research consistently identifies several recurring challenges introduced by decomposition:

- **Operational and orchestration complexity:** functionality that used to reside in a single process must now be coordinated across many independently deployed containers, often via container orchestration platforms such as Kubernetes [4], [5].
- **Network and communication overhead:** inter-process calls over the network replace fast in-memory function calls, introducing latency, the risk of cascading failures, and reduced fault tolerance under load [4], [6].
- **Data consistency challenges:** each microservice is generally expected to own a private database, which complicates cross-service transactions and requires eventual-consistency strategies rather than a single ACID-compliant data store [4].
- **Debugging and observability difficulty:** microservice systems are large, dynamic, concurrent distributed systems in which instances are created and destroyed on demand, and a single user request can traverse a long, asynchronous chain of service calls, making root-cause diagnosis substantially harder than in a monolith [4], [5].

D. Performance Comparisons

Performance studies reinforce that microservice adoption is not a universal improvement. Blinowski et al. [6] implemented a reference application in both monolithic and microservice form across two languages (Java and C#) and three deployment environments, and found that on a single machine a monolith consistently outperforms its microservice-based counterpart due to the added inter-process communication overhead and only once request volume and available scaling resources increase, does the microservice version become competitive. Similarly, Christian et al. [2] conducted a systematic literature review and reported that in concurrency testing scenarios monolithic architectures frequently exhibit better raw throughput than equivalent microservice implementations, even though microservices retain the advantage in modularity, independent scalability, and maintainability. These findings are consistent with real-world cases such as Amazon Prime Video and portions of Istio's own control plane, both of which moved components back from a microservice to a monolithic design once distributed overhead outweighed the benefits of decomposition for the specific workload involved [2].

E. RESEARCH QUESTIONS

This research aims to investigate the following research question:

- How can an AI-driven self-healing framework combining graph neural network-based anomaly detection and reinforcement learning-based recovery improve the reliability and operational resilience of microservice systems?

To answer this research question, the following objectives are addressed:

- **O1**: Develop a GGNN-Deep SVDD anomaly detection model capable of accurately identifying abnormal execution traces in cloud-native microservice systems.
- **O2**: Design a reinforcement learning-based recovery policy that recommends appropriate recovery actions for different fault conditions.
- **O3**: Implement an adaptive recovery executor with automated recovery verification to enable autonomous fault remediation.
- **O4**: Integrate an LLM-based explainability module to generate human-readable explanations of detected faults and recovery decisions.
- **O5**: Evaluate the proposed framework through offline benchmarking and online deployment in the Train-Ticket microservice system.

III. RELATED WORK

The literature review conducted for this project examined recent research into AI-driven self-healing agents for microservice architectures. Across the studies investigated, a clear shift emerges away from manual, rule-based fault management and toward learning-based autonomous recovery. As microservice deployments grow in scale and complexity, human-driven monitoring and incident response become increasingly time-consuming and error-prone, motivating sustained research interest in automated fault detection, diagnosis, and remediation.

A. Key Research Findings

1) Reinforcement learning enables adaptive, cost-aware recovery: Samir et al. frames self-healing as a control problem formalized as a Markov Decision Process, in which system states encode runtime conditions, actions correspond to recovery operations, and reward functions balance recovery speed against performance impact and operational cost [7]. QMConn applies Q-learning to remediate Kubernetes configuration and access-control faults, selecting among service restarts, configuration updates, and rollbacks, and reports a mean time to recovery of roughly 13 minutes per error for single-fault scenarios and 23.75 minutes under simultaneous multi-error conditions. More recent work extends this controller paradigm by using large language models to convert operational data (logs, metrics) into structured fault representations that are then used by a deep reinforcement learning agent [8], improving both recovery accuracy and efficiency relative to rule-based baselines.

2) Graph-based and telemetry-driven models are recurring design patterns: Investigating fault detection, diagnosis, and root-cause literature, a consistent pattern emerges, representing services and their interactions as graphs, and combining this structural view with temporal telemetry. GAL-MAD [9] combines Graph Attention Networks, which model inter-service dependencies, with Bi-LSTM layers, which capture temporal dynamics, to detect anomalies arising from complex cross-service interactions, and further uses SHAP-based explainability to attribute detected anomalies to specific services and metrics. Similar graph-structured reasoning underlies root-cause analysis approaches such as TraceDiag [10], which prunes large service-dependency graphs using reinforcement learning and causal analysis, and MicroRCA [11], which ranks likely faulty services with Personalized PageRank over an attributed graph of service and host-level metrics. This graph and telemetry-driven pattern recurs throughout the reviewed literature and is a key design influence for this projects approach to representing system state.

3) Verification remains an under explored but necessary factor to learning-based recovery: It is important that all learning-based recovery agents can be verified for correctness. ESBMC-AI [12] integrates bounded model checking with LLM-generated code repair, re-verifying each fix before acceptance, while Ehrig et al. [13] models self-healing behaviour as algebraic graph transformations to verify properties such as deadlock-freeness at design time, while Carwehl et al. [14] extends verification to runtime in order to handle requirements that change during execution. Complementing these design-time guarantees, chaos engineering approaches such as CHESS [15] proactively inject faults to evaluate resilience under realistic runtime conditions rather than static models. A systematic mapping study [16] of 21 primary studies found that only around 23.81% explicitly address self-healing, and that the majority of existing approaches remain reactive rather than proactive, underscoring that verification and validation of autonomous recovery is comparatively immature relative to the detection and decision-making techniques above.

4) Challenges: Despite the promising results reported across this literature, several recurring limitations were identified:

- Heavy reliance on offline training and controlled fault-injection experiments, limiting robustness to unseen failure modes and evolving workloads.
- Recovery decisions are frequently executed without formal safety or correctness guarantees, introducing risk in production deployments.
- Explainability of learned models remains limited outside of specific systems (e.g., GAL-MAD's SHAP-based attribution), making it difficult to audit why a given recovery action was chosen.
- Weak integration between the three pipeline stages, anomaly detection, root cause analysis, and recovery planning, with most studies optimizing one stage in isolation rather than the full pipeline.
- Verification and validation techniques (formal methods, chaos engineering) are comparatively under-adopted relative to the maturity of detection and RL-based decision-making methods.

5) Summary: Taken together, the reviewed literature demonstrates that reinforcement learning, graph-based modelling, and multi-source telemetry analysis have emerged as the dominant techniques in self-healing agents. However, the literature also makes clear that trustworthy, verifiable autonomous self-healing, particularly under safety constraints and unseen operating conditions, remains an open problem, and directly motivates the structured, verifiable reasoning approach adopted in this project.

IV. METHODOLOGY

A. Overall Framework

This paper proposes an AI-driven self-healing framework for Train-Ticket microservice systems that integrates graph-based anomaly detection, reinforcement learning-based recovery, adaptive recovery execution, automated verification, and large language model (LLM) explainability into a unified operational pipeline. Unlike conventional observability platforms that primarily detect abnormal system behaviour, the proposed framework autonomously detects faults, recommends appropriate recovery actions, executes the selected recovery strategy, verifies service restoration, and generates human-readable explanations for operators.

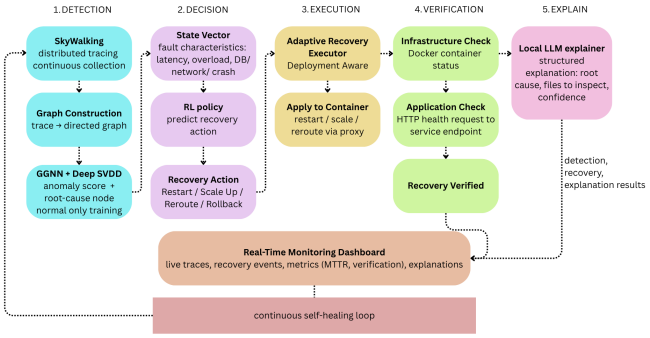

Fig. 1. Overall architecture of the proposed self-healing framework.

Figure 1 illustrates the overall architecture of the proposed framework. Distributed execution traces are continuously collected from the Train-Ticket microservice benchmark using Apache SkyWalking. Each execution trace is transformed into a graph representation and analysed by a Gated Graph Neural Network (GGNN) with Deep Support Vector Data Description (Deep SVDD) to identify anomalous behaviour. When an anomaly is detected, the framework constructs a numerical state representation describing the current system condition. The state vector is provided to a reinforcement learning policy that predicts the most appropriate recovery action. The selected action is executed by an adaptive recovery executor and subsequently verified using infrastructure-level and application-level health checks. Finally, a locally deployed LLM generates a structured explanation describing the detected fault, recovery decision, and recommended code-level investigation.

B. Distributed Trace Collection

The proposed framework employs Apache SkyWalking as the observability platform for collecting distributed execution traces from the Train-Ticket microservice benchmark. Every microservice is instrumented utilizing the Skywalking Java agent, enabling end-to-end request propagation to be captured automatically without requiring modifications to the application source code.

Each user request generates a distributed trace composed of multiple spans. Every span records execution metadata, including service name, endpoint, execution duration, parent-child relationship, timestamps, and error status. Compared with traditional system metrics, distributed traces preserve both temporal and structural dependencies between microservices, allowing complex service interactions to be analysed more effectively.

The collected traces are retrieved through the SkyWalking GraphQL API. After retrieval, unnecessary metadata is removed and the remaining information is transformed into graph structures suitable for graph neural network processing. Each span represents a graph node, while invocation relationships between spans define the graph edges.

C. GGNN-Based Anomaly Detection

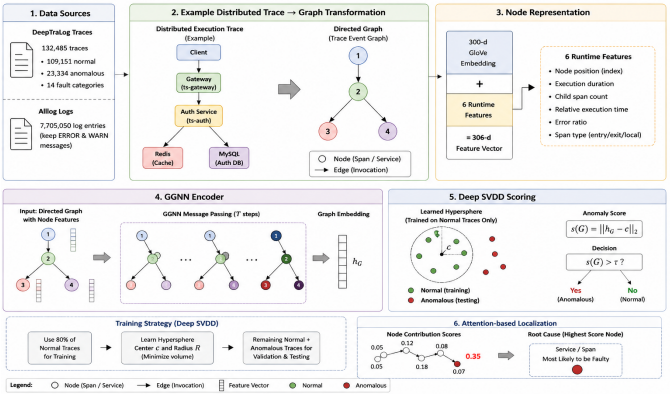

Fig. 2. Overall Pipeline of GGNN-based anomaly detection.

The anomaly detection component is based on the DeepTraLog framework, which represents distributed execution traces as directed graphs [17]. The model was trained using the DeepTraLog dataset comprising 132,485 distributed traces, including 109,151 normal and 23,334 anomalous traces spanning 14 fault categories [17]. To enrich graph semantics, trace information was combined with 7,705,050 log entries from the alllog dataset. Semantic node representations were generated using pre-trained 300-dimensional GloVe embeddings, while only ERROR and WARN log messages were retained to reduce noise [18].

Deep Support Vector Data Description (Deep SVDD) is a one-class anomaly detection method that learns a compact representation of normal system behaviour. Rather than learning to distinguish between normal and anomalous samples, Deep SVDD is trained exclusively on normal data to learn a

hypersphere enclosing normal graph embeddings in the latent feature space. During inference, the Euclidean distance between a graph embedding and the learned hypersphere centre is computed as the anomaly score. Embeddings that lie close to the centre are classified as normal, whereas those outside the learned boundary are identified as anomalous [19]. Following the Deep SVDD learning strategy described above, the detector was trained exclusively on normal execution traces. Specifically, 80% of the normal traces were used for training, while the remaining normal traces together with all anomalous traces were reserved for validation and testing. Unlike conventional supervised anomaly detection, Deep SVDD formulates anomaly detection as a one-class classification problem, where the objective is to learn a compact representation of normal system behaviour. Training solely on normal traces prevents anomalous samples from influencing the learned hypersphere boundary, allowing previously unseen or rare failures to be detected as deviations from normal operational patterns. This strategy also improves the model's ability to generalise to fault types that may not be represented in the training data.

Each execution trace is transformed into a directed graph in which graph nodes correspond to service spans and graph edges represent invocation relationships. Every node is represented by a 306-dimensional feature vector comprising a 300-dimensional GloVe embedding together with six runtime features describing node position, execution duration, child span count, relative execution time, error ratio, and span type.

The graph is processed using a Gated Graph Neural Network (GGNN), which performs iterative message passing to capture long-range dependencies between service invocations [20]. Compared with conventional Graph Convolutional Networks (GCNs), the GGNN more effectively models sequential interactions commonly observed in distributed execution traces [21].

Graph embeddings produced by the GGNN are evaluated using the Deep SVDD objective, which assigns anomaly scores based on the distance between graph embeddings and a learned hypersphere centre. Traces whose anomaly scores exceed a dynamically determined threshold are classified as anomalous.

To facilitate fault localisation, an attention-based localisation mechanism estimates the contribution of each graph node to the overall anomaly score. The node with the highest contribution score is identified as the most likely root-cause service and forwarded to the recovery module.

D. Reinforcement Learning Recovery Policy

Following anomaly detection, the framework constructs a numerical state representation describing the operational condition of the affected microservice. The state incorporates multiple fault characteristics, including anomaly severity, service availability, response latency, overload conditions, network degradation, database failures, asynchronous execution faults, and error ratios.

The recovery policy was trained offline using 23,334 labelled anomalous traces extracted from the DeepTraLog GraphData dataset, covering 14 representative fault categories. The dataset was randomly divided into 18,667 training samples (80%) and 4,667 testing samples (20%). The policy network was first initialized using supervised learning to establish an initial mapping between fault states and recovery actions before being further optimized using the REINFORCE policy-gradient algorithm. During runtime, the trained policy is used solely for inference without additional online learning.

The recovery action space consists of four candidate actions: `RESTART`, `SCALE_UP`, `REROUTE`, and `ROLLBACK`.

For each detected anomaly, the policy outputs a probability distribution over the candidate actions, and the action with the highest probability is selected as the recommended recovery strategy.

E. Adaptive Recovery and Verification

The recommended recovery action is executed by an adaptive recovery executor that considers the current deployment state before initiating recovery. Rather than blindly executing the selected action, the executor evaluates service availability and deployment conditions to determine whether an alternative execution strategy would improve service continuity.

For example, when the recovery policy recommends restarting a failed service, the executor first checks whether a healthy backup instance is available. If a backup exists, incoming traffic is redirected through the Nginx reverse proxy, allowing recovery to proceed through rerouting instead of restarting the failed container. Resource scaling dynamically adjusts Docker container resources, while restart operations recover unavailable services by restarting the corresponding container.

Following execution, infrastructure-level verification confirms that the target container is operational, while application-level verification performs HTTP health checks against the recovered service endpoint. Recovery is considered successful only when both verification stages complete successfully. For each recovery event, the framework records detection latency, executed recovery action, verification status, and Mean Time to Recovery (MTTR), which are displayed through the monitoring dashboard.

F. LLM-Based Recovery Explanation

To improve the interpretability of recovery decisions, the framework integrates a locally deployed Qwen2.5-7B-Instruct instruction-tuned language model as an explainability component [22]. The LLM is not involved in anomaly detection or recovery selection. Instead, it receives structured information generated by the preceding components, including the affected service, anomaly score, baseline score, predicted root cause, system state, and selected recovery action.

Based on this information, the LLM generates a structured explanation containing the likely root cause, justification for the identified service, relevant files or classes for inspection, recommended code-level modifications, runtime recovery recommendations, and an estimated confidence score. Because the LLM functions solely as an explainability layer, incorrect or hallucinated responses cannot influence anomaly detection

or recovery execution, ensuring that all operational decisions remain deterministic.

V. EXPERIMENTAL SETUP

This section describes the experimental environment, deployment configuration, fault injection scenarios, and evaluation metrics used to assess the proposed self-healing framework.

A. Experimental Environment

The proposed framework was evaluated using the Train-Ticket benchmark, an open-source cloud-native microservice application that simulates an online railway ticket booking system. The benchmark consists of multiple independently deployed services communicating through RESTful APIs, making it suitable for evaluating distributed tracing, anomaly detection, and autonomous recovery in realistic microservice environments.

All services were deployed using Docker Compose on a Windows-based workstation. Apache SkyWalking was integrated into the deployment to provide distributed tracing and runtime observability. Each microservice was instrumented using the SkyWalking Java agent, enabling execution traces to be collected transparently without modifying the application source code.

The proposed self-healing framework was implemented as an external monitoring service that continuously retrieved distributed traces from SkyWalking through its GraphQL API. Docker was used to execute recovery operations, including container restart and resource scaling, while HTTP health checks were performed to verify successful service restoration following recovery.

B. Model Configuration

The deployed anomaly detector corresponds to the proposed GGNN-Deep SVDD model described in Section III. The model was trained using the DeepTraLog benchmark containing 132,485 distributed execution traces, including 109,151 normal and 23,334 anomalous traces spanning 14 representative fault categories. Following the Deep SVDD learning strategy, only normal execution traces were used during training, while anomalous traces were reserved for validation and testing.

The reinforcement learning recovery policy was trained offline using the 23,334 labelled anomalous traces extracted from the DeepTraLog GraphData dataset. The dataset was randomly divided into 18,667 training samples (80%) and 4,667 testing samples (20%). The policy network was first initialized using supervised learning before being refined using the REINFORCE policy-gradient algorithm. During deployment, the trained policy was used exclusively for inference without additional online learning.

Prior to deployment, multiple anomaly detection architectures and recovery policies were evaluated offline. The best-performing GGNN-Deep SVDD detector and the supervised pretraining followed by REINFORCE recovery policy (A3) were selected for online evaluation.

C. Online Deployment

Following offline training, the selected anomaly detector and recovery policy were deployed within the Train-Ticket benchmark to evaluate end-to-end autonomous self-healing under live operating conditions [23].

Distributed execution traces collected by Apache SkyWalking were continuously analysed using the trained GGNN-Deep SVDD anomaly detector. When anomalous behaviour was identified, the detected fault characteristics were converted into a numerical state representation and provided to the trained reinforcement learning policy, which predicted one of four recovery actions: Restart, Scale Up, Reroute, or Rollback.

The recommended recovery action was executed by the adaptive recovery executor. Following execution, infrastructure-level verification was performed by checking Docker container status, while application-level verification was conducted through HTTP health requests to the affected service endpoint. Recovery was considered successful only when both verification stages completed successfully.

D. Fault Injection

To evaluate the proposed framework under representative runtime failures, multiple fault scenarios were injected into the Train-Ticket deployment. The evaluated faults were selected to represent common operational issues encountered in cloud-native microservice systems while exercising different recovery strategies.

Faults were injected into running Docker containers using resource constraints, network emulation, and controlled container failures. Following fault injection, distributed traces were continuously collected through Apache SkyWalking and analysed by the proposed framework. Recovery actions were then executed automatically without manual intervention, except for rollback operations, which required explicit operator approval due to their potential impact on application consistency.

Each fault scenario was repeated ten times under identical experimental conditions to evaluate the consistency and robustness of the proposed framework.

E. Evaluation Metrics

The effectiveness of the proposed framework was evaluated from both anomaly detection and autonomous recovery perspectives.

The anomaly detection component was evaluated using standard classification metrics, including Precision, Recall, F1-score, and Area Under the Receiver Operating Characteristic Curve (AUC-ROC). These metrics were used to compare the proposed GGNN-Deep SVDD detector against alternative anomaly detection approaches evaluated during the offline experiments.

The online self-healing capability was evaluated using the following operational metrics:

- **Detection Success Rate** — percentage of injected faults successfully detected.

- **Recovery Success Rate** — percentage of detected faults successfully recovered.
- **Detection Latency** — time elapsed between fault occurrence and anomaly detection.
- **Mean Time to Recovery (MTTR)** — time elapsed between anomaly detection and successful recovery verification.
- **Recovery Verification** — confirmation of successful recovery using both Docker container status and HTTP health checks.

Together, these metrics provide a comprehensive assessment of both the detection capability and the operational effectiveness of the proposed self-healing framework.

VI. RESULTS & DISCUSSION

To comprehensively evaluate the proposed AI-driven self-healing framework, three complementary experiments were conducted. First, the proposed GGNN-based anomaly detector was evaluated offline using the DeepTraLog benchmark. Second, the reinforcement learning recovery policy was independently evaluated to assess recovery action selection. Finally, the complete framework was deployed within the Train-Ticket benchmark to evaluate end-to-end autonomous fault detection and recovery under representative runtime failures.

A. Offline Anomaly Detection Performance

The first experiment evaluates the effectiveness of the proposed anomaly detector independently from the recovery module. Four models were compared using the same trace dataset: Isolation Forest [24]; a Graph Convolutional Network (GCN) with Deep SVDD [21]; a GGNN trained using mixed normal and abnormal traces [20]; and the proposed GGNN trained using only normal traces. Their detection performance is reported in Table I.

TABLE I
4-MODEL ABLATION STUDY RESULTS (⋆ = PROPOSED MODEL)

Model	AUC-ROC	Precision	Recall	F1
M1: Isolation Forest	0.917	0.917	0.846	0.880
M2: GCN + SVDD	0.854	0.678	0.895	0.772
M3: GGNN mixed training	0.869	0.756	0.913	0.827
M4: GGNN normal-only ⋆	**0.947**	**0.968**	0.749	**0.845**

The proposed GGNN trained exclusively on normal execution traces achieved the highest AUC-ROC (0.947) and Precision (0.968) among all evaluated models. Although its recall was lower than the mixed-training approaches, the significantly higher precision is desirable for autonomous self-healing systems because false-positive detections may trigger unnecessary recovery actions and temporarily disrupt service availability.

Compared with Isolation Forest and the GCN-based detector, the proposed GGNN more effectively captured structural dependencies between distributed service invocations, enabling more reliable discrimination between normal and anomalous execution traces. These results justify selecting the proposed GGNN-Deep SVDD model as the anomaly detector deployed in the online self-healing framework.

Interestingly, the proposed GGNN trained exclusively on normal traces outperformed the GGNN trained on mixed normal and anomalous data. This observation suggests that learning a compact representation of healthy system behaviour is more effective for one-class anomaly detection than explicitly modelling known fault patterns. Because production microservice environments continuously evolve and new failure modes may emerge over time, relying solely on labelled anomalies may reduce generalisation. The superior performance of the normal-only model therefore supports the adoption of the Deep SVDD training strategy for autonomous self-healing systems.

B. Model Ablation Study

To understand the contribution of each design decision, an ablation study was performed by progressively replacing components of the proposed detector. The same evaluation metrics were used for comparison.

The results demonstrate that each architectural improvement contributes to the overall detection capability. Isolation Forest provides a strong statistical baseline but cannot capture structural dependencies within distributed traces. Replacing conventional graph convolution with gated graph neural networks improves the representation of execution dependencies between spans. Furthermore, training solely on normal traces enables Deep SVDD to construct a more compact representation of healthy system behaviour, resulting in the highest AUROC and precision among all evaluated models.

These findings justify the selection of the proposed GGNN trained on normal traces as the anomaly detector used in the online self-healing framework.

C. Reinforcement Learning Policy Evaluation

The reinforcement learning recovery policy was evaluated independently to determine its ability to recommend appropriate recovery actions before deployment.

Three recovery strategies were compared, A1-Rule-Based Recovery, A2- REINFORCE, A3- Supervised Pretraining + REINFORCE. Performance was evaluated using the held-out testing dataset consisting of 4,667 labelled anomalous traces.

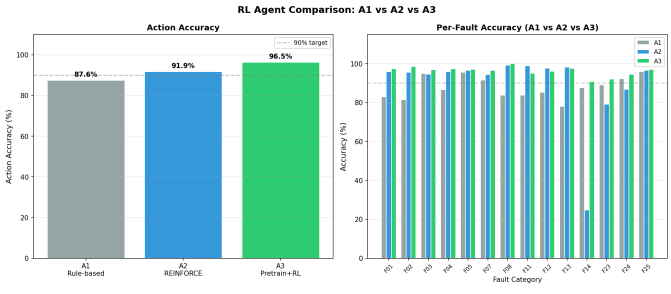

Fig. 3. Description of the figure.

As illustrated in Fig. 3, the proposed recovery policy (A3) achieved the highest overall recovery-action selection accuracy

of 96.5%, outperforming both the rule-based baseline (87.6%) and the REINFORCE-only policy (91.9%).

The improvement demonstrates that supervised pretraining provides an effective initialization of the recovery policy, while subsequent REINFORCE optimisation further refines recovery decisions through reward-driven learning [25].

The per-fault evaluation further shows that the proposed policy consistently achieved greater than 90% action-selection accuracy across all evaluated fault categories, approaching perfect accuracy for several fault types. These results indicate that the learned policy generalises effectively across diverse microservice failures and provides a reliable basis for online autonomous recovery.

D. Online Self-Healing Evaluation

Following offline validation, the complete self-healing framework was deployed within the Train-Ticket benchmark to evaluate end-to-end fault detection and recovery under representative runtime failures.

Four representative fault categories were selected because they exercise different recovery strategies:

- CPU Overload → Scale Up
- Network Delay → Reroute
- Service Crash → Restart/Reroute
- Database Error → Restart
- DB Error → Restart

Each fault scenario was repeated ten times under identical experimental conditions to evaluate the consistency and robustness of the proposed framework. For every trial, the following metrics were recorded:

- Detection success
- Recovery success
- Detection latency
- Mean Time To Recovery (MTTR)
- Executed Recovery Action

Rollback was implemented as a manual recovery option requiring operator approval because automatic rollback may revert valid deployments or introduce schema incompatibilities. Consequently, rollback was not included in the automated evaluation.

TABLE II
FAULT DETECTION AND RECOVERY PERFORMANCE

Fault Type	**Det. Rate**	**Rec. Rate**	**Det. Time (s)**	**MTTR (s)**	**Recovery Action**
CPU Overload	10/10	10/10	4.61	11.88	`SCALE_UP`
Network Delay	10/10	10/10	1.36	9.19	`REROUTE`
Service Crash	10/10	10/10	2.50	18.19	`RESTART/REROUTE`
DB Error	10/10	10/10	1.00	4.05	`RESTART`

All detection and recovery rates are 100% (10/10).

The proposed framework achieved a 100% detection rate and 100% recovery success rate across all evaluated fault scenarios.

CPU overload faults were successfully mitigated through dynamic resource scaling, while network delay faults were recovered by rerouting traffic to healthy service instances. Database failures were resolved through automatic container restart.

For service crashes, the reinforcement learning policy recommends Restart. However, the adaptive recovery executor performs Reroute whenever a healthy backup instance is available, thereby reducing service interruption. Otherwise, the original restart recommendation is executed. This deployment-aware adaptation enables the framework to select recovery strategies according to the current runtime environment rather than following a fixed recovery policy.

The observed detection latency ranged from 1.00 s to 4.61 s, while recovery completed within 4.05 s to 18.19 s, demonstrating that the framework can rapidly restore service availability under representative runtime failures.

E. Explainability of Recovery Decisions

To improve operator understanding of recovery events, the proposed framework incorporates a locally deployed Qwen2.5-7B-Instruct instruction-tuned language model as an explainability component.

For each detected anomaly, the model receives structured information including the affected service, anomaly score, baseline score, predicted root cause, system state, and selected recovery action. Based on these inputs, the LLM generates human-readable explanations describing the likely root cause, justification for the identified service, recommended code-level inspection points, runtime recovery recommendations, and an estimated confidence score.

Qualitative inspection showed that the generated explanations were generally consistent with both the detected fault category and the selected recovery strategy. For example, CPU overload events were associated with resource contention and scaling recommendations, while service crashes were explained in terms of container availability and restart or rerouting strategies.

Because the LLM functions solely as an explainability layer, it does not participate in anomaly detection or recovery selection. Consequently, incorrect or hallucinated responses cannot influence the operational behaviour of the self-healing framework.

F. Discussion

The experimental results demonstrate that combining graph-based anomaly detection with reinforcement learning enables effective autonomous self-healing for cloud-native microservice systems.

The offline experiments show that representing distributed execution traces as graphs allows the proposed GGNN-Deep SVDD detector to capture structural relationships between service invocations more effectively than statistical and conventional graph-based baselines. The resulting high precision reduces false-positive detections, an important characteristic

for self-healing systems where unnecessary recovery actions may themselves disrupt service availability.

The reinforcement learning evaluation further demonstrates that combining supervised pretraining with policy-gradient optimisation produces more reliable recovery decisions than either manually designed rules or reinforcement learning alone. Rather than relying on predefined recovery mappings, the learned policy adapts recovery decisions according to the observed runtime state.

The online deployment confirms that integrating anomaly detection, recovery selection, adaptive execution, and automated verification enables consistent end-to-end fault management under representative runtime failures. In particular, the adaptive recovery executor improves operational flexibility by considering the current deployment state before executing the selected recovery action. For example, service crash faults may be recovered through rerouting rather than restarting whenever healthy backup instances are available.

Finally, the addition of a local LLM improves the interpretability of recovery events without affecting operational safety. Because recovery decisions remain exclusively controlled by the trained reinforcement learning policy and adaptive recovery executor, explanation errors cannot alter the behaviour of the self-healing framework.

Overall, the proposed framework demonstrates that integrating graph neural networks, reinforcement learning, deployment-aware recovery execution, and explainable AI provides a practical foundation for intelligent self-healing in cloud-native microservice environments.

G. Comparison with State-Of-Art Anomaly Detection Methods

Table III compares the proposed model with representative anomaly detection methods reported in the DeepTraLog study. The benchmark results are reproduced from the original publication for comparison purposes.

TABLE III
COMPARISON WITH STATE-OF-THE-ART ANOMALY DETECTION METHODS

Method	Precision	Recall	F1
TraceAnomaly [26]	0.742	0.205	0.321
MultimodalTrace [27]	0.591	0.776	0.671
DeepLog [28]	0.608	0.948	0.741
LogAnomaly [29]	0.415	**0.977**	0.582
DeepTraLog [17]	0.930	0.978	**0.954**
Proposed GGNN–Deep SVDD	**0.968**	0.749	0.845

The proposed GGNN-Deep SVDD anomaly detector was further compared with the original DeepTraLog framework, which represents the closest related work and was evaluated on the same benchmark dataset. Table III summarises the reported performance.

Although the proposed model achieved a higher precision (0.968 vs. 0.930), indicating fewer false-positive detections, DeepTraLog reported higher recall (0.978 vs. 0.749) and F1-score (0.954 vs. 0.845). The higher precision of the proposed model is desirable for autonomous self-healing systems, where false-positive anomaly detection may trigger unnecessary recovery actions and temporarily disrupt service availability.

While DeepTraLog focuses primarily on anomaly detection and root-cause localisation, the proposed work extends this capability by integrating reinforcement learning-based recovery, adaptive recovery execution, automated recovery verification, and LLM-based explainability into a unified end-to-end self-healing framework. Consequently, the contribution of this work lies not only in competitive anomaly detection performance but also in providing autonomous fault management from detection through recovery and explanation.

H. Limitations

Although the proposed framework demonstrates promising performance for autonomous fault detection and recovery, several limitations remain.

First, the online evaluation was conducted using the Train-Ticket benchmark deployed on a single-host Docker environment. While Train-Ticket is widely adopted for microservice research, it does not fully capture the scale, deployment complexity, and dynamic behaviour of production cloud platforms. Additional evaluation on large-scale Kubernetes clusters would provide a more comprehensive assessment of the framework's scalability and robustness.

Second, only four representative runtime fault scenarios were evaluated during online deployment, namely CPU overload, network delay, service crash, and database failure. Although the reinforcement learning policy was trained using traces covering fourteen fault categories, the remaining fault types were not experimentally validated because of hardware limitations and the computational cost associated with large-scale fault injection.

Third, the reinforcement learning policy is trained offline and remains fixed during deployment. Consequently, the framework cannot automatically adapt its recovery strategy when previously unseen fault patterns or substantially different workloads are encountered. Periodic retraining is therefore required to maintain recovery performance as the operational environment evolves.

Finally, the integrated large language model serves solely as an explainability component and was evaluated qualitatively rather than quantitatively. Although this design prevents incorrect explanations from affecting recovery decisions, future work should investigate systematic evaluation methodologies for measuring explanation quality and usefulness to system operators.

VII. CONCLUSION

This paper presented an AI-driven self-healing framework for cloud-native microservice systems that integrates GGNN-based anomaly detection, reinforcement learning-based recovery, adaptive recovery execution, automated verification, and LLM-based explainability into a unified operational pipeline.

Experimental evaluation demonstrated that the proposed GGNN-Deep SVDD detector achieved superior anomaly detection performance compared with alternative approaches,

while the proposed reinforcement learning policy achieved the highest recovery-action selection accuracy among the evaluated recovery strategies. Online deployment within the Train-Ticket benchmark further demonstrated reliable end-to-end autonomous fault detection and recovery across representative runtime failures, achieving 100% detection and recovery success in the evaluated scenarios.

Overall, the results indicate that combining graph neural networks, reinforcement learning, and deployment-aware recovery execution provides an effective foundation for intelligent self-healing in cloud-native microservice systems.

A. Future Work

Future work will investigate deployment on large-scale Kubernetes clusters, evaluation across the remaining DeepTraLog fault categories, and support for cascading multi-service failures. The recovery policy will also be extended through online reinforcement learning, allowing recovery decisions to adapt continuously to evolving workloads and previously unseen failure patterns. In addition, future research will explore quantitative evaluation of LLM-generated explanations using expert assessment and user-centred studies to better understand their effectiveness in supporting system operators.

VIII. GENERATIVE AI DISCLOSURE

Gemini was employed in a strictly limited fashion, used only for aid with Overleaf, concept clarification and grammar / language improvement. LLMs were not used for any ideas, results or conclusions.

REFERENCES

[1] V. Velepucha and P. Flores, "A survey on microservices architecture: Principles, patterns and migration challenges," *IEEE Access*, vol. 11, pp. 88 339–88 358, 2023.

[2] J. Christian, Steven, A. Kurniawan, and M. S. Anggreainy, "Analyzing microservices and monolithic systems: Key factors in architecture, development, and operations," in *2023 6th Int. Conf. of Computer and Informatics Eng. (IC2IE)*, Jakarta, Indonesia, 2023, pp. 64–69.

[3] G. Pathak and M. Singh, "A review of cloud microservices architecture for modern applications," in *2023 World Conf. on Communication & Computing (WCONF)*, 2023, pp. 1–7.

[4] G. Liu, B. Huang, Z. Liang, M. Qin, H. Zhou, and Z. Li, "Microservices: Architecture, container, and challenges," in *2020 IEEE 20th Int. Conf. Softw. Quality, Reliability and Security Companion (QRS-C)*, 2020, pp. 629–635.

[5] F. Al-Doghman, N. Moustafa, I. Khalil, N. Sohrabi, Z. Tari, and A. Y. Zomaya, "AI-enabled secure microservices in edge computing: Opportunities and challenges," *IEEE Trans. Services Comput.*, vol. 16, no. 2, pp. 1485–1504, 2023.

[6] G. Blinowski, A. Ojdowska, and A. Przybyłek, "Monolithic vs. microservice architecture: A performance and scalability evaluation," *IEEE Access*, vol. 10, pp. 20 357–20 374, 2022.

[7] A. Samir, H. Dagenborg, and D. Johansen, "QMConn: A self-healing controller for microservices using q-learning and markov decision processes," in *2024 IEEE/ACM 17th Int. Conf. on Utility and Cloud Computing (UCC)*, Sharjah, UAE, 2024, pp. 389–398.

[8] Z. Yang, Y. Jin, J. Liu, and X. Xu, "An intelligent fault self-healing mechanism for cloud ai systems via integration of large language models and deep reinforcement learning," 2025.

[9] L. Akmeemana, C. Attanayake, H. Faiz, and S. Wickramanayake, "GAL-MAD: Towards explainable anomaly detection in microservice applications using graph attention networks," *arXiv preprint arXiv:2504.00058*, 2025.

[10] R. Ding *et al.*, "Tracediag: Adaptive, interpretable, and efficient root cause analysis on large-scale microservice systems," 2023.

[11] L. Wu, J. Tordsson, E. Elmroth, and O. Kao, "Microrca: Root cause localization of performance issues in microservices," in *NOMS 2020 - 2020 IEEE/IFIP Network Operations and Management Symposium*, 2020, pp. 1–9.

[12] N. Tihanyi, Y. Charalambous, R. Jain, M. A. Ferrag, and L. C. Cordeiro, "A new era in software security: Towards self-healing software via large language models and formal verification," in *2025 IEEE/ACM International Conference on Automation of Software Test (AST)*, 2025, pp. 136–147.

[13] H. Ehrig, C. Ermel, O. Runge, A. Bucchiarone, and P. Pelliccione, "Formal analysis and verification of self-healing systems," 2010, pp. 139–153.

[14] M. Carwehl, T. Vogel, G. N. Rodrigues, and L. Grunske, "Runtime verification of self-adaptive systems with changing requirements," in *2023 IEEE/ACM 18th Symposium on Software Engineering for Adaptive and Self-Managing Systems (SEAMS)*, 2023, pp. 104–114.

[15] M. A. Naqvi, S. Malik, M. Astekin, and L. Moonen, "On evaluating self-adaptive and self-healing systems using chaos engineering," in *2022 IEEE International Conference on Autonomic Computing and Self-Organizing Systems (ACSOS)*, 2022, pp. 1–10.

[16] M. Filho, E. Pimentel, W. Pereira, P. H. M. Maia, and M. I. Cortés, "Self-adaptive microservice-based systems - landscape and research opportunities," in *2021 International Symposium on Software Engineering for Adaptive and Self-Managing Systems (SEAMS)*, 2021, pp. 167–178.

[17] C. Zhang, X. Peng, C. Sha, K. Zhang, Z. Fu, X. Wu, Q. Lin, and D. Zhang, "DeepTraLog: Trace-log combined microservice anomaly detection through graph-based deep learning," in *Proc. 44th Int. Conf. Softw. Eng. (ICSE)*, Pittsburgh, PA, USA, 2022, pp. 623–634.

[18] J. Pennington, R. Socher, and C. D. Manning, "GloVe: Global vectors for word representation," in *Proc. 2014 Conf. Empir. Methods Nat. Lang. Process. (EMNLP)*, Doha, Qatar, 2014, pp. 1532–1543.

[19] L. Ruff, R. A. Vandermeulen, N. Görnitz, L. Deecke, S. A. Siddiqui, A. Binder, E. Müller, and M. Kloft, "Deep one-class classification," in *Proc. 35th Int. Conf. Mach. Learn. (ICML)*, Stockholm, Sweden, 2018, pp. 4393–4402.

[20] Y. Li, D. Tarlow, M. Brockschmidt, and R. Zemel, "Gated graph sequence neural networks," in *Proc. 4th Int. Conf. Learn. Represent. (ICLR)*, San Juan, Puerto Rico, 2016.

[21] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *Proc. 5th Int. Conf. Learn. Represent. (ICLR)*, Toulon, France, 2017.

[22] Qwen Team, "Qwen2.5 technical report," *arXiv preprint arXiv:2412.15115*, 2025.

[23] X. Zhou *et al.*, "Fault analysis and debugging of microservice systems: Industrial survey, benchmark system, and empirical study," *IEEE Trans. Softw. Eng.*, vol. 47, no. 2, pp. 243–260, 2021.

[24] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in *Proc. 8th IEEE Int. Conf. Data Mining (ICDM)*, Pisa, Italy, 2008, pp. 413–422.

[25] R. J. Williams, "Simple statistical gradient-following algorithms for connectionist reinforcement learning," *Machine Learning*, vol. 8, no. 3–4, pp. 229–256, 1992.

[26] P. Liu, H. Xu, Q. Ouyang, R. Jiao, Z. Chen, S. Zhang, J. Yang, L. Mo, J. Zeng, W. Xue, and D. Pei, "Unsupervised detection of microservice trace anomalies through service-level deep bayesian networks," in *Proc. 31st IEEE Int. Symp. Softw. Reliability Eng. (ISSRE)*, 2020, pp. 48–58.

[27] S. Nedelkoski, J. S. Cardoso, and O. Kao, "Anomaly detection from system tracing data using multimodal deep learning," in *Proc. 12th IEEE Int. Conf. Cloud Computing (CLOUD)*, 2019, pp. 179–186.

[28] M. Du, F. Li, G. Zheng, and V. Srikumar, "DeepLog: Anomaly detection and diagnosis from system logs through deep learning," in *Proc. 2017 ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*, 2017, pp. 1285–1298.

[29] W. Meng, Y. Liu, Y. Zhu, S. Zhang, D. Pei, Y. Liu, Y. Chen, R. Zhang, S. Tao, P. Sun, and R. Zhou, "LogAnomaly: Unsupervised detection of sequential and quantitative anomalies in unstructured logs," in *Proc. 28th Int. Joint Conf. Artif. Intell. (IJCAI)*, 2019, pp. 4739–4745.